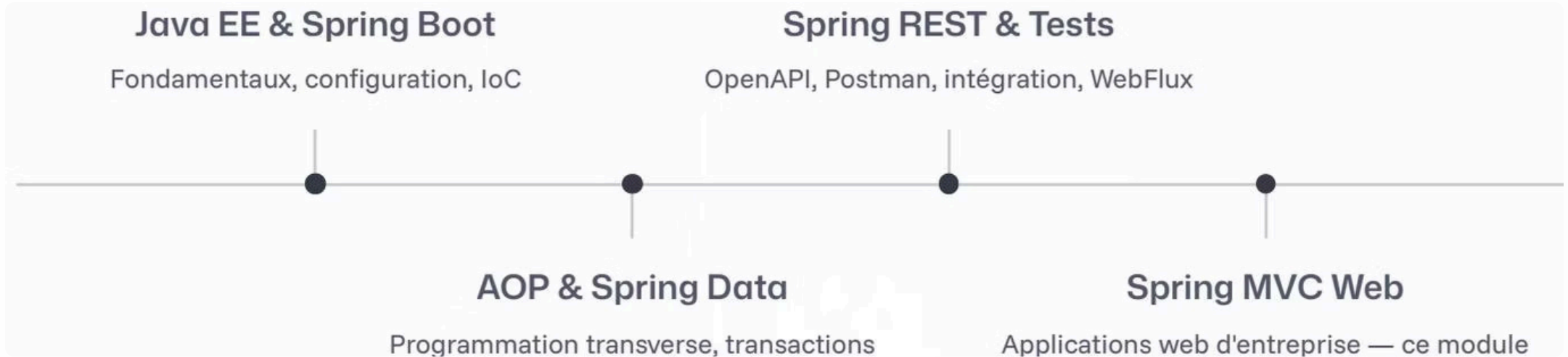




Spring MVC

Parcours de formation



Vous abordez ici la partie **interfaces utilisateur côté serveur**, là où la logique métier rencontre le rendu visuel.

Objectifs du module

À l'issue de ce module, vous serez capable de concevoir et développer des applications web complètes avec Spring MVC, de la réception des requêtes HTTP au rendu HTML dans le navigateur.



Ce que vous allez maîtriser



Comprendre

L'architecture MVC et son implémentation dans Spring Framework



Implémenter

Des contrôleurs Spring MVC robustes et maintenables



Maîtriser

Thymeleaf pour le templating de vues dynamiques



Valider

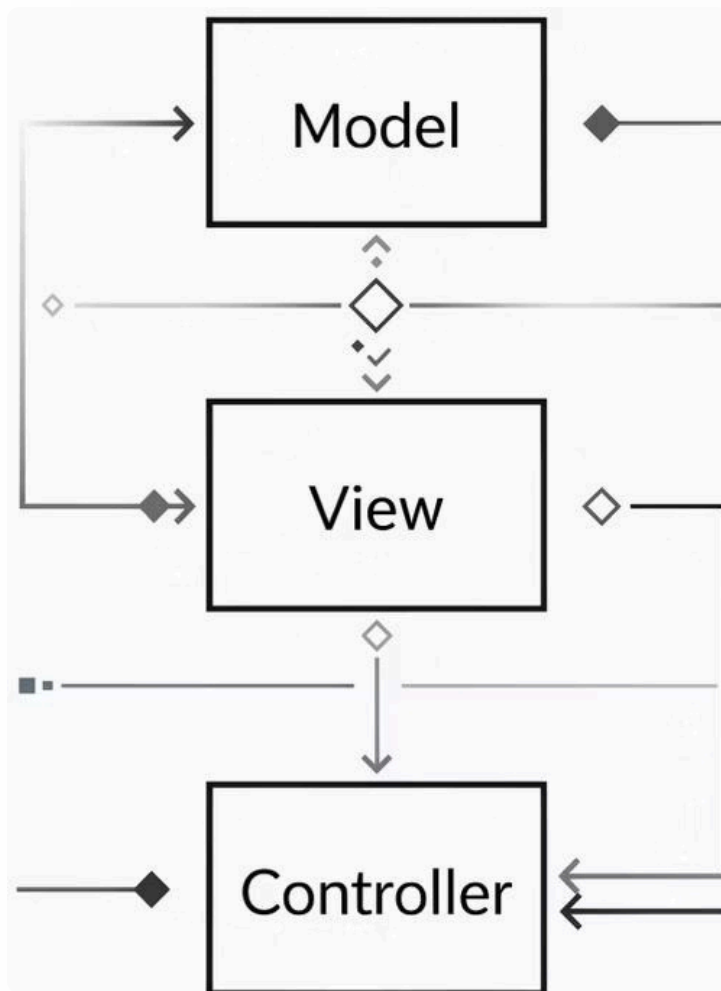
Les formulaires avec Bean Validation



Gérer

Les ressources statiques : CSS, JS et images

Le patron MVC



Séparation des responsabilités

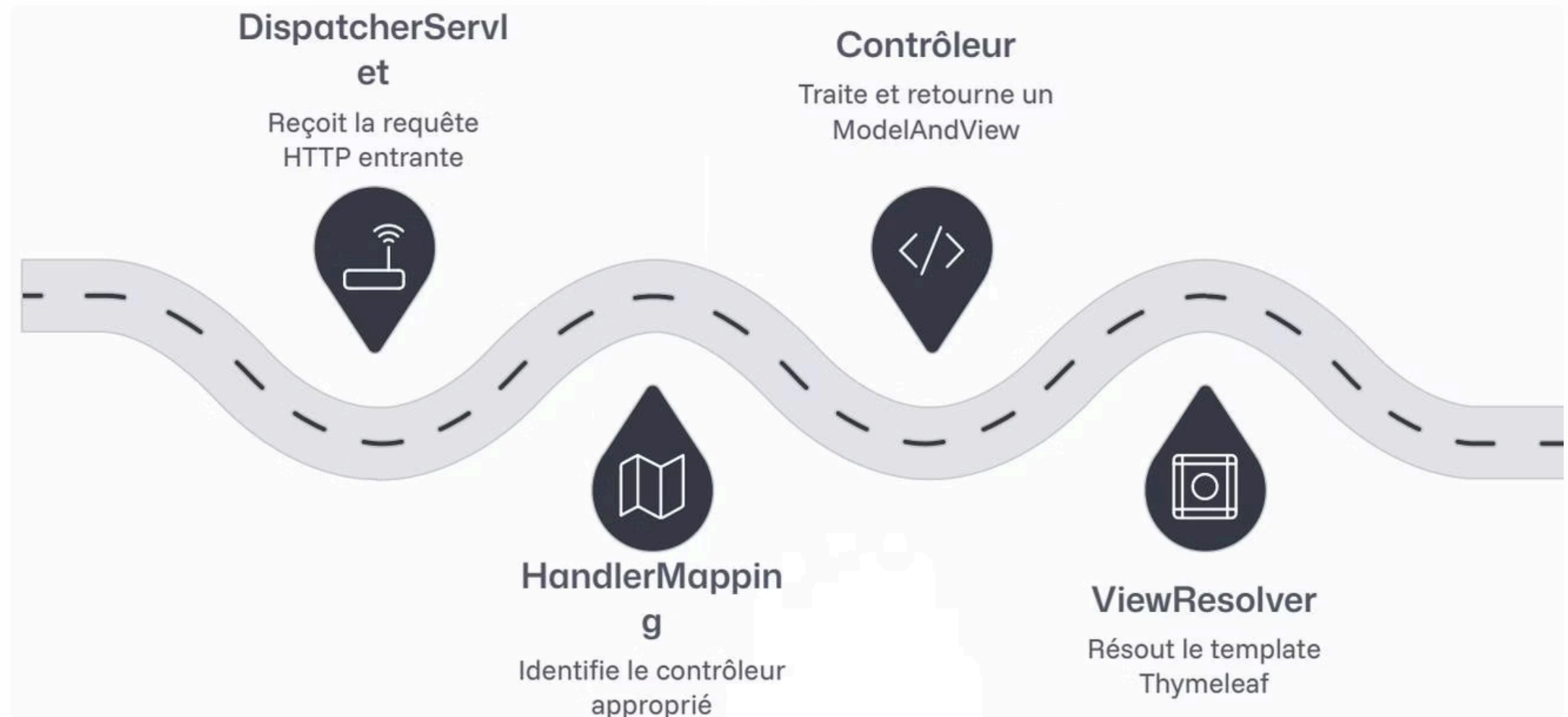
Le patron **Model-View-Controller** divise l'application en trois composants distincts :

- **Modèle** — données et logique métier
- **Vue** — rendu et présentation
- **Contrôleur** — orchestration des interactions

Cette séparation favorise la maintenabilité, la testabilité et la réutilisabilité du code.

Le DispatcherServlet

Dans Spring, l'architecture MVC s'articule autour d'un composant central : le **DispatcherServlet**. Il reçoit toutes les requêtes HTTP entrantes et les délègue aux composants appropriés — c'est le *Front Controller Pattern*.



Cycle de vie d'une requête



Ce flux découplé permet de changer de moteur de template sans modifier une seule ligne de code métier.

Les composants clés

1

DispatcherServlet

Point d'entrée unique de toutes les requêtes HTTP.
Configure automatiquement les beans Spring MVC au démarrage.

2

HandlerMapping

Résout quelle méthode de contrôleur doit traiter une requête donnée en fonction de l'URL et de la méthode HTTP.

3

Controller

Bean annoté `@Controller` qui contient la logique de traitement et prépare le modèle de données.

4

ViewResolver

Traduit un nom de vue logique (ex: "home") en ressource physique (`/templates/home.html`).

Auto-configuration Spring Boot

Ce que Spring Boot configure automatiquement

- Serveur Tomcat embarqué sur le port 8080
- DispatcherServlet mappé sur /
- ViewResolver Thymeleaf vers /templates/
- Ressources statiques depuis /static/
- Conversion des types et liaison de données
- Gestion des erreurs par défaut

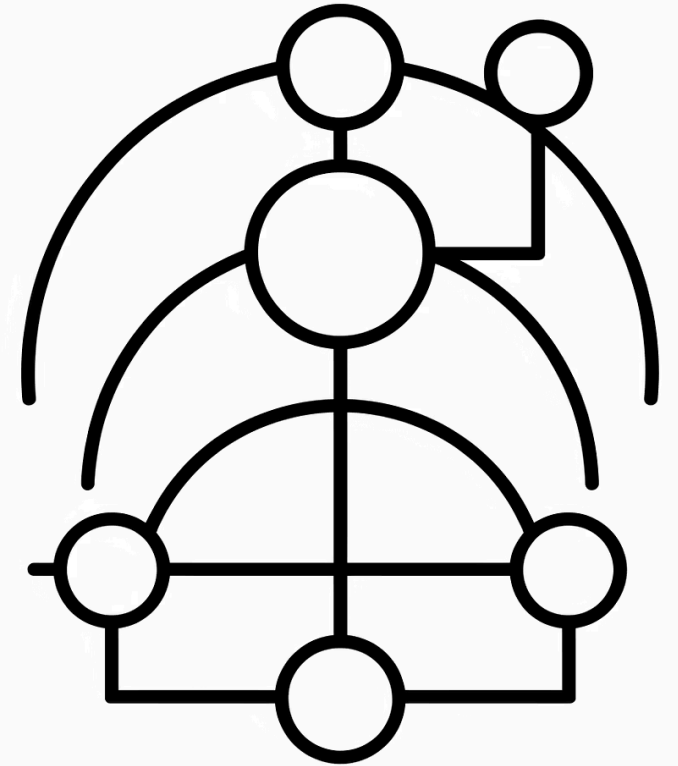
✅ Zéro configuration XML requise avec Spring Boot !

```
<dependency>
  <groupId>
    org.springframework.boot
  </groupId>
  <artifactId>
    spring-boot-starter-web
  </artifactId>
</dependency>

<!-- Pour Thymeleaf -->
<dependency>
  <groupId>
    org.springframework.boot
  </groupId>
  <artifactId>
    spring-boot-starter-thymeleaf
  </artifactId>
</dependency>
```

Les Contrôleurs Spring MVC

Le cœur de votre application web



Rôle du contrôleur

Un contrôleur Spring MVC est un bean annoté `@Controller`. Contrairement à `@RestController`, il retourne des **noms de vues** et non des données JSON sérialisées.

→ Interceptor

Reçoit les requêtes HTTP entrantes via les annotations de mapping

→ Traiter

Appelle la logique métier et prépare les données pour la vue

→ Retourner

Renvoie le nom logique de la vue à afficher

Anatomie d'un contrôleur

@Controller

@RequestMapping("/produits")

```
public class ProduitController {
```

@Autowired

```
private ProduitService service;
```

@GetMapping

```
public String liste(Model model) {
```

```
    model.addAttribute("produits",
```

```
        service.findAll());
```

```
    return "produits/liste";
```

```
}
```

@GetMapping("/{id}")

```
public String detail(
```

```
    @PathVariable Long id,
```

```
    Model model) {
```

```
    model.addAttribute("produit",
```

```
        service.findById(id));
```

```
    return "produits/detail";
```

```
}
```

```
}
```

Points clés

@RequestMapping

Définit le préfixe URL commun à toutes les méthodes du contrôleur

Model

Dictionnaire clé-valeur qui transporte les données vers le template

String retourné

Nom logique de la vue que le ViewResolver résout en template physique

Annotations de mapping HTTP

Annotation	Méthode HTTP	Usage typique	Exemple
@GetMapping	GET	Afficher une ressource	/produits
@PostMapping	POST	Soumettre un formulaire	/produits/nouveau
@PutMapping	PUT	Mise à jour complète	/produits/{id}
@DeleteMapping	DELETE	Suppression	/produits/{id}
@PatchMapping	PATCH	Mise à jour partielle	/produits/{id}

 Dans une application web avec formulaires HTML, seuls GET et POST sont disponibles nativement. Spring MVC propose `HiddenHttpMethodFilter` pour simuler PUT et DELETE.

Extraction des données de la requête

1

@PathVariable

Extrait une valeur depuis le chemin URL. Ex :

`@GetMapping("/{id}")` avec `@PathVariable Long id` extrait l'ID depuis `/produits/42`.

2

@RequestParam

Extrait un paramètre de la query string. Ex :

`@RequestParam(defaultValue="0") int page` pour la pagination avec `?page=2`.

3

@ModelAttribute

Lie automatiquement les données d'un formulaire HTML à un objet Java. Spring peuple chaque champ depuis les paramètres POST.

4

@RequestHeader

Extrait une valeur depuis les en-têtes HTTP. Utile pour lire `Accept-Language` ou des tokens personnalisés.

Model vs ModelAndView

Approche avec Model (recommandée)

```
@GetMapping("/dashboard")
public String dashboard(
    Model model) {

    model.addAttribute(
        "utilisateur",
        securityService.currentUser());
    model.addAttribute(
        "statistiques",
        statsService.getStats());

    return "dashboard/index";
}
```

Approche avec ModelAndView

```
@GetMapping("/dashboard")
public ModelAndView dashboard() {

    ModelAndView mav =
        new ModelAndView(
            "dashboard/index");

    mav.addObject(
        "utilisateur",
        securityService.currentUser());

    return mav;
}
```

 Préférez **Model** pour sa lisibilité. Utilisez **ModelAndView** quand le nom de la vue doit être déterminé dynamiquement.



Gestion des Vues

ViewResolver, Thymeleaf et résolution de templates

Résolution des vues

Lorsque le contrôleur retourne un nom de vue, le **ViewResolver** trouve la ressource correspondante. Avec Spring Boot et Thymeleaf, la convention est simple :

Nom logique retourné

"produits/liste"

Fichier physique résolu

src/main/resources/templates/produits/liste.html

Ce mécanisme de *convention over configuration* simplifie considérablement le développement.

Configuration du ViewResolver

application.properties

```
# Préfixe des templates
spring.thymeleaf.prefix=
  classpath:/templates/

# Suffixe des templates
spring.thymeleaf.suffix=.html

# Encodage
spring.thymeleaf.encoding=UTF-8

# Cache désactivé en dev
spring.thymeleaf.cache=false

# Mode de rendu
spring.thymeleaf.mode=HTML
```

Structure recommandée

```
src/main/resources/
├── templates/
│   ├── layout/
│   │   ├── base.html
│   │   ├── navbar.html
│   │   └── footer.html
│   ├── produits/
│   │   ├── liste.html
│   │   ├── detail.html
│   │   └── formulaire.html
│   ├── index.html
│   └── error.html
├── static/
├── css/
├── js/
└── images/
```

Redirections et Forwards

redirect:

Envoie une réponse HTTP 302. Le navigateur effectue une nouvelle requête GET. L'URL change dans la barre d'adresse.

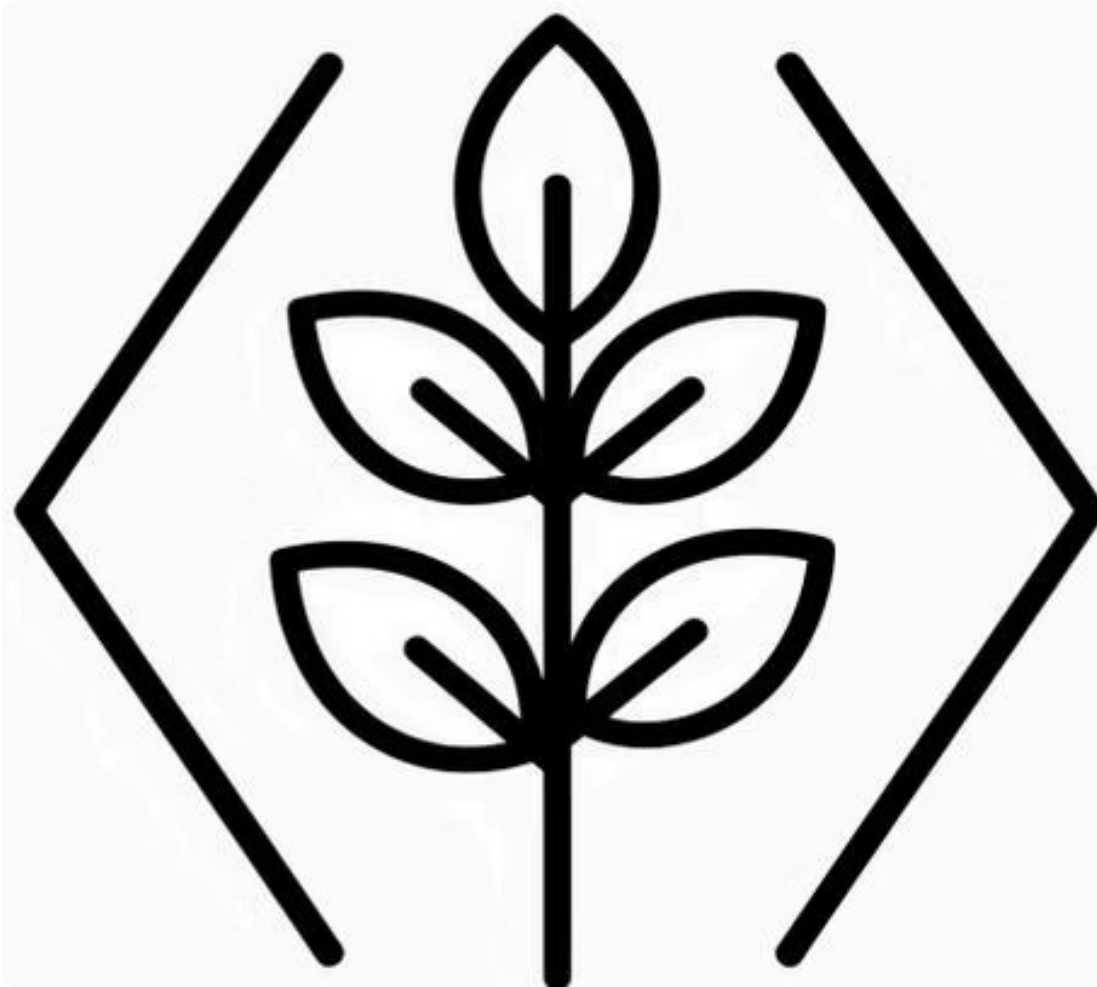
```
return "redirect:/produits";
```

forward:

Transmet la requête en interne sans que le navigateur ne le sache. L'URL ne change pas. Utile pour partager un modèle.

```
return "forward:/produits/liste";
```

⚠ Toujours utiliser `redirect:` après un POST réussi pour éviter la resoumission de formulaire lors d'un rechargement de page — c'est le pattern **PRG (Post/Redirect/Get)**.



Thymeleaf

Le moteur de template recommandé par l'équipe Spring

Natural Template Engine

Qu'est-ce que Thymeleaf ?

Thymeleaf est un **Natural Template Engine** : ses fichiers HTML sont des fichiers HTML valides, ouverts directement dans un navigateur sans serveur. Les directives sont des attributs `th:` invisibles aux navigateurs non-Thymeleaf.

Avantage majeur : les designers peuvent travailler sur les templates sans environnement Spring opérationnel.

Les expressions Thymeleaf

- `${...}` — Variable du Model
- `*{...}` — Sélection sur objet lié
- `#{...}` — Message i18n
- `@{...}` — Construction d'URL
- `~{...}` — Référence à un fragment

Attributs Thymeleaf essentiels

<!-- Affichage de texte -->

```
<h1 th:text="${produit.nom}">Nom par défaut</h1>
```

<!-- Lien dynamique -->

```
<a th:href="@{/produits/{id}(id=${produit.id})}">Voir le détail</a>
```

<!-- Boucle sur une liste -->

```
<tr th:each="p : ${produits}">
```

```
<td th:text="${p.nom}"></td>
```

```
<td th:text="${p.prix}"></td>
```

```
</tr>
```

<!-- Condition -->

```
<span th:if="${produit.enStock}">Disponible</span>
```

```
<span th:unless="${produit.enStock}">Rupture de stock</span>
```

Attributs Thymeleaf – Référence

- | | | | |
|---|--|---|---|
|  | th:text
Remplace le contenu texte du tag HTML |  | th:href / th:src
URLs dynamiques pour liens et images |
|  | th:each
Itération sur collections |  | th:if / th:unless
Rendu conditionnel |
|  | th:field
Liaison bidirectionnelle formulaire-objet |  | th:object
Sélection d'objet de référence pour les formulaires |

Fragments et Layouts Thymeleaf

Définition d'un fragment

```
<!-- templates/layout/base.html -->
<html xmlns:th=
  "http://www.thymeleaf.org">
<head>
  <title th:fragment="titre">
    Mon App
  </title>
  <link th:href="@{/css/app.css}"
    rel="stylesheet"/>
</head>
<body>
  <nav th:fragment="navbar">
    <!-- Navigation -->
  </nav>
  <div th:fragment="contenu">
    <!-- Remplacé par les pages -->
  </div>
  <footer th:fragment="footer">
    <!-- Pied de page -->
  </footer>
</body>
```

Pourquoi les fragments ?

Le système de fragments permet de découper les templates en **composants réutilisables** : entête, pied de page, navigation, alertes.

- Évite la duplication de code HTML
- Garantit la cohérence visuelle
- Référencé via `~{layout/base :: navbar}`

Validation des Formulaires

Bean Validation (JSR-380) avec Hibernate Validator

Email

Required field

Password

Required field

Company

Invalid format

Invalid format

Approche déclarative

Spring MVC s'intègre avec **Bean Validation (JSR-380)**. Au lieu d'écrire du code de validation impératif, vous placez des annotations directement sur votre DTO. Spring gère la collecte des erreurs et leur transmission à la vue.

Sur le DTO

Annotations `@NotBlank`, `@Size`,
`@Email`, `@Min` sur les champs

Dans le contrôleur

Paramètre `@Valid` + `BindingResult`
pour déclencher et capturer les
erreurs

Dans la vue

Affichage via `th:errors` et
`#fields.hasErrors()`

Classe DTO annotée

```
public class ProduitDto {  
  
    @NotBlank(message = "Le nom est obligatoire")  
    @Size(min = 2, max = 100,  
    message = "Entre 2 et 100 caractères")  
    private String nom;  
  
    @NotNull(message = "Le prix est requis")  
    @DecimalMin(value = "0.01",  
    message = "Prix doit être positif")  
    private BigDecimal prix;  
  
    @Size(max = 500,  
    message = "Max 500 caractères")  
    private String description;  
  
    @NotNull  
    @Min(value = 0, message = "Stock non négatif")  
    private Integer quantiteStock;  
  
    @Email(message = "Email invalide")  
    private String emailFournisseur;  
}
```

Annotations de validation disponibles

Annotation	Contrainte
@NotNull	Non nul
@NotBlank	Non vide (String)
@NotEmpty	Non vide (collection)
@Size	Taille min/max
@Min / @Max	Valeur numérique
@Email	Format email
@Pattern	Expression régulière
@Past / @Future	Contrainte de date
@Positive	Nombre positif

Contrôleur avec validation

```
@PostMapping("/nouveau")
public String creer(
    @Valid @ModelAttribute("produit")
    ProduitDto dto,
    BindingResult bindingResult,
    Model model) {

    if (bindingResult.hasErrors()) {
        // Réaffiche le formulaire
        return "produits/formulaire";
    }

    produitService.save(dto);
    // Pattern PRG
    return "redirect:/produits";
}

@GetMapping("/nouveau")
public String afficherFormulaire(
    Model model) {
    model.addAttribute(
        "produit", new ProduitDto());
    return "produits/formulaire";
}
```

Règle critique

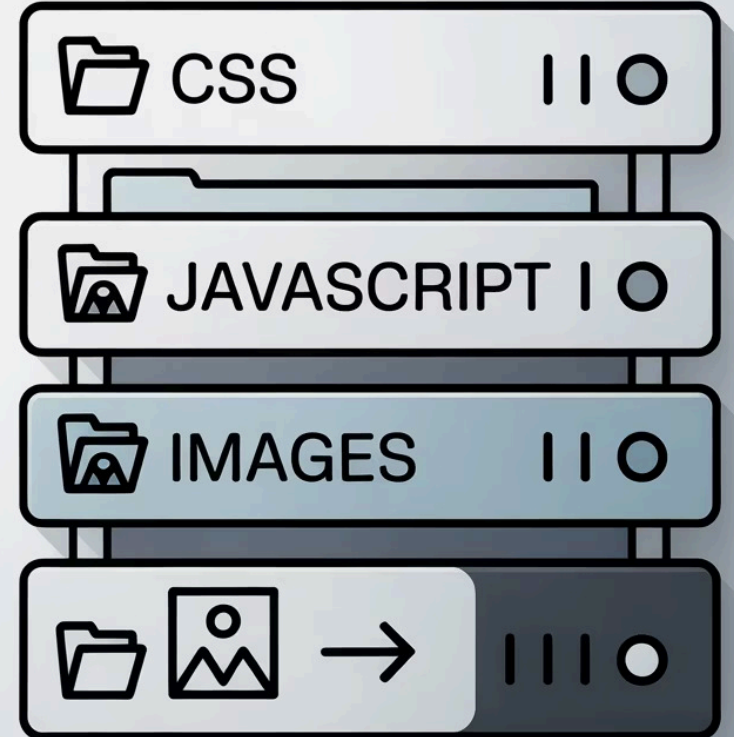
⚠ Le paramètre `BindingResult` doit être placé **immédiatement après** le paramètre annoté `@Valid`, sans aucun autre paramètre entre les deux. Sinon Spring MVC lance une exception.

Pattern PRG

Après un POST réussi, toujours rediriger avec `redirect:` pour éviter la resoumission du formulaire lors d'un rechargement de page.

Ressources Statiques

CSS, JavaScript, images et WebJars



Gestion automatique des ressources

Spring Boot configure automatiquement le **ResourceHttpRequestHandler**. Toute ressource placée dans les répertoires spéciaux est accessible directement via HTTP.

1

/META-INF/resources/

Priorité maximale. Ressources packagées dans des JAR tiers (WebJars).

2

/resources/

Deuxième priorité. Ressources à distribuer avec l'application.

3

/static/

Le répertoire le plus utilisé pour CSS, JS et images.

4

/public/

Priorité la plus basse. Ressources publiquement accessibles.

 Un fichier `src/main/resources/static/css/app.css` est accessible à l'URL `http://localhost:8080/css/app.css`.

WebJars – Dépendances Frontend via Maven

Dépendance Maven

```
<dependency>
  <groupId>org.webjars</groupId>
  <artifactId>bootstrap</artifactId>
  <version>5.3.2</version>
</dependency>

<!-- Résolution auto de version -->
<dependency>
  <groupId>org.webjars</groupId>
  <artifactId>
    webjars-locator-core
  </artifactId>
</dependency>
```

Inclusion dans Thymeleaf

```
<!-- Avec webjars-locator (auto) -->
<link rel="stylesheet"
  th:href="@{
    /webjars/bootstrap/
    css/bootstrap.min.css}"
/>

<script
  th:src="@{
    /webjars/bootstrap/
    js/bootstrap.bundle.min.js}">
</script>
```

Les WebJars éliminent la gestion manuelle des fichiers CSS/JS et permettent de versionner les dépendances frontend avec Maven.

Cache Busting – Versioning des ressources

Configuration

```
# application.properties (prod)
spring.web.resources
    .chain.strategy
    .content.enabled=true

spring.web.resources
    .chain.strategy
    .content.paths=/**

# URL générée automatiquement :
# /css/app-a1b2c3d4.css
```

Utilisation dans Thymeleaf

```
<!-- Dans le template -->
<link th:href="@{/css/app.css}"
      rel="stylesheet"/>

<!-- HTML rendu en production -->
<link href=
  "/css/app-a1b2c3d4.css"
  rel="stylesheet"/>
```

✅ Aucun changement de code requis — Spring gère le versioning automatiquement !

Gestion des erreurs HTTP

Page générique

Créez `templates/error.html` pour intercepter toutes les erreurs. Accès aux détails via `${status}`, `${error}`, `${message}`.

Pages spécifiques

Créez `templates/error/404.html` et `templates/error/500.html`. Spring Boot les détecte automatiquement.

@ControllerAdvice

Pour une gestion avancée, utilisez `@ControllerAdvice` avec des méthodes `@ExceptionHandler` pour centraliser la gestion des exceptions métier.

Internationalisation (i18n)

Fichiers de messages

```
# messages.properties (défaut)
produit.titre.liste=
    Liste des produits
produit.champ.nom=Nom
produit.btn.ajouter=Ajouter

# messages_en.properties
produit.titre.liste=Product List
produit.champ.nom=Name
produit.btn.ajouter=Add

# application.properties
spring.messages.basename=messages
spring.messages.encoding=UTF-8
```

Utilisation dans Thymeleaf

```
<h1 th:text="
    #{produit.titre.liste}">
    Liste des produits
</h1>

<label th:text="
    #{produit.champ.nom}">
    Nom
</label>

<button type="submit"
    th:text="
    #{produit.btn.ajouter}">
    Ajouter
</button>
```

 Combinez i18n avec Bean Validation en définissant les messages d'erreur dans `messages.properties` pour une validation complètement localisée.

Dépendances et configuration du TP

pom.xml

```
<!-- Spring MVC + Tomcat -->
spring-boot-starter-web

<!-- Thymeleaf -->
spring-boot-starter-thymeleaf

<!-- Bean Validation -->
spring-boot-starter-validation

<!-- DevTools -->
spring-boot-devtools
(scope: runtime, optional)

<!-- H2 en mémoire -->
h2 (scope: runtime)
```

application.properties

```
server.port=8080
spring.thymeleaf.cache=false
spring.thymeleaf.encoding=UTF-8

spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
spring.datasource.url=
jdbc:h2:mem:testdb

spring.jpa.hibernate.ddl-auto=
create-drop
spring.jpa.show-sql=true

logging.level.
org.springframework.web=DEBUG
```

 Activez le mode DEBUG Spring Web pour voir dans la console quels contrôleurs sont mappés et quelles vues sont résolues.

Récapitulatif du module

01

Architecture MVC

DispatcherServlet orchestre toutes les requêtes. HandlerMapping, Controller, ViewResolver sont les acteurs clés.

02

Contrôleurs Spring

@Controller + annotations de mapping HTTP. L'objet Model transporte les données vers la vue.

03

Thymeleaf

Natural templates, expressions `${}` `@{}` `#{}`, attributs `th:each` `th:if` `th:field`, fragments réutilisables.

04

Validation

Bean Validation avec @Valid et BindingResult, affichage des erreurs via `th:errors`, pattern PRG.

05

Ressources statiques

Répertoire `/static/`, WebJars pour les dépendances frontend, versioning automatique pour le cache busting.